Computer System Architecture

(THIRD EDITION)

Morris Mano

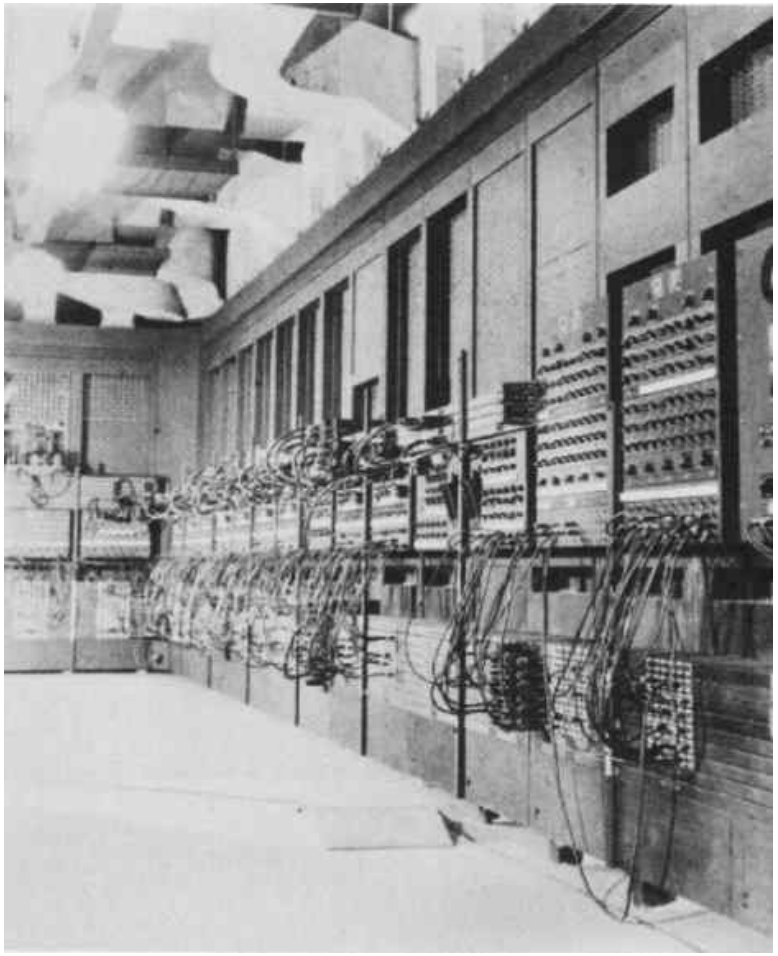
PRENTICE HALL

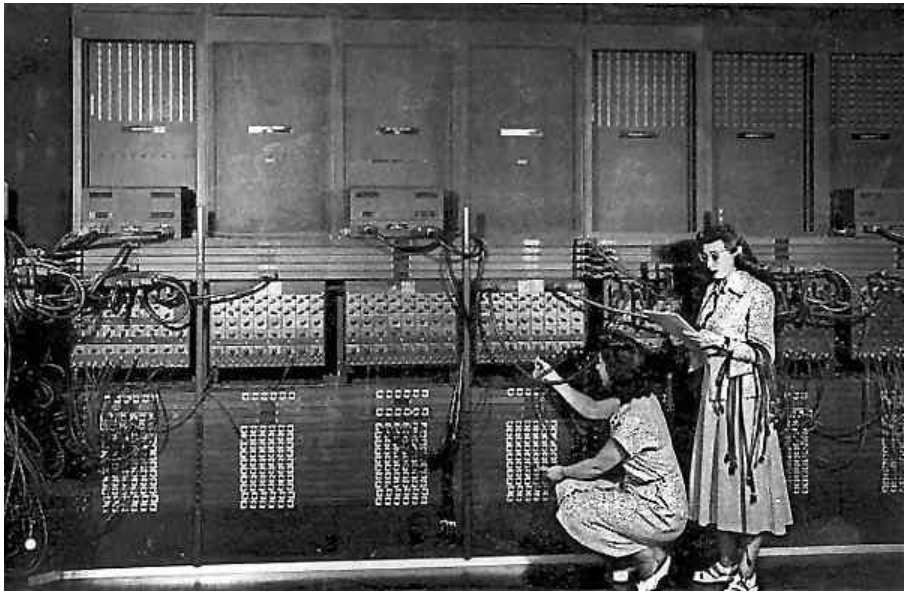
Class Overview

- Contents
  - Basic Computer Hardware Architecture
  - CPU, Memory, I/O System Design
  - Advanced Topics: Pipeline, Multiprocessor
- Related Subject
  - Digital Logic (논리설계), Discrete Mathematics (이산수학)
- Chapter Outline
  - Ch. 1 ~ Ch. 4: Digital Computer Hardware Architecture
  - Ch. 5 ~ Ch. 7: Basic Computer Design
  - Ch. 8 ~ Ch. 10: CPU Architecture and Pipeline Processing
  - Ch. 11 ~ Ch. 12: I/O and Memory Architecture
  - Ch. 13: Multiprocessor
- Homework
  - Solve the selected number of problems
  - Report of Survey for given theme
- Reference
  - Morris Mano, Computer Systems Architecture, Prentice-Hall, 3rd Ed.
  - John Hennessy, David Patterson, Computer Architecture: a Quantitative approach, Morgan Kaufmann. 3rd Ed.

최초의 컴퓨터 ?

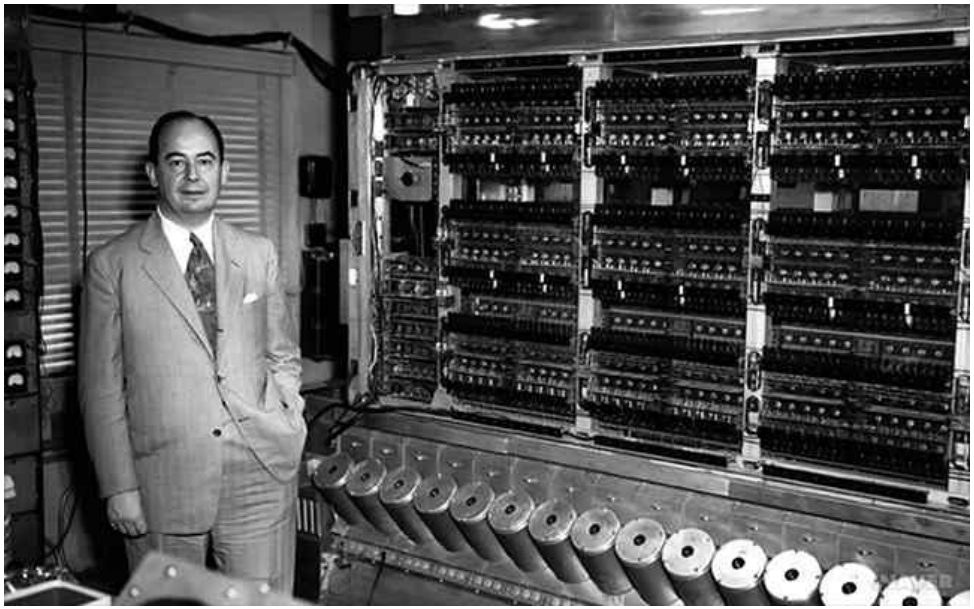
- ENIAC (1946)

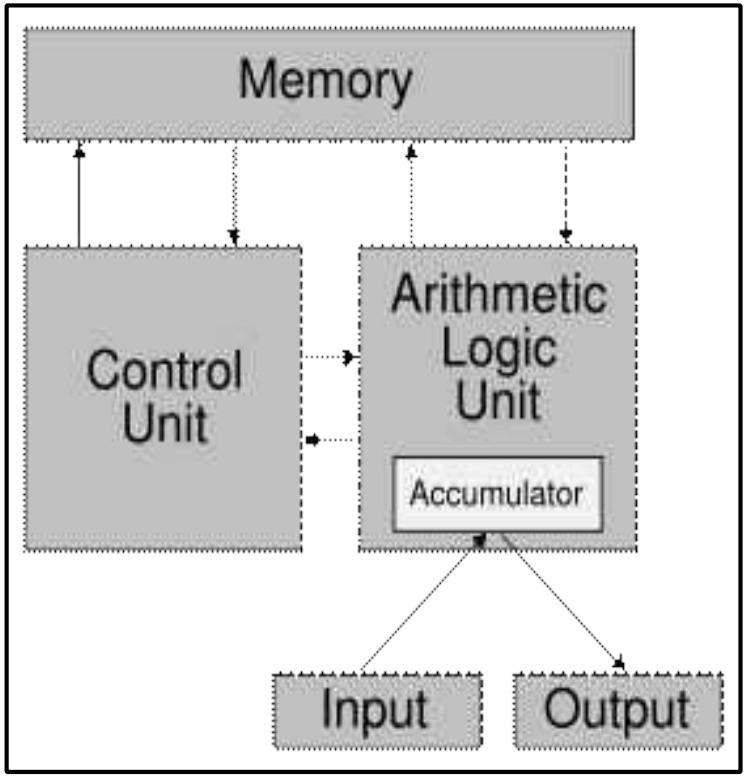




Von Neumann Architecture

- John von Neumann & EDVAC





Quiz

Question 2

TechLogoQuiz

An urban legend suggested this company's logo was inspired by the manner in which early tech visionary Alan Turing died.

A) Intel

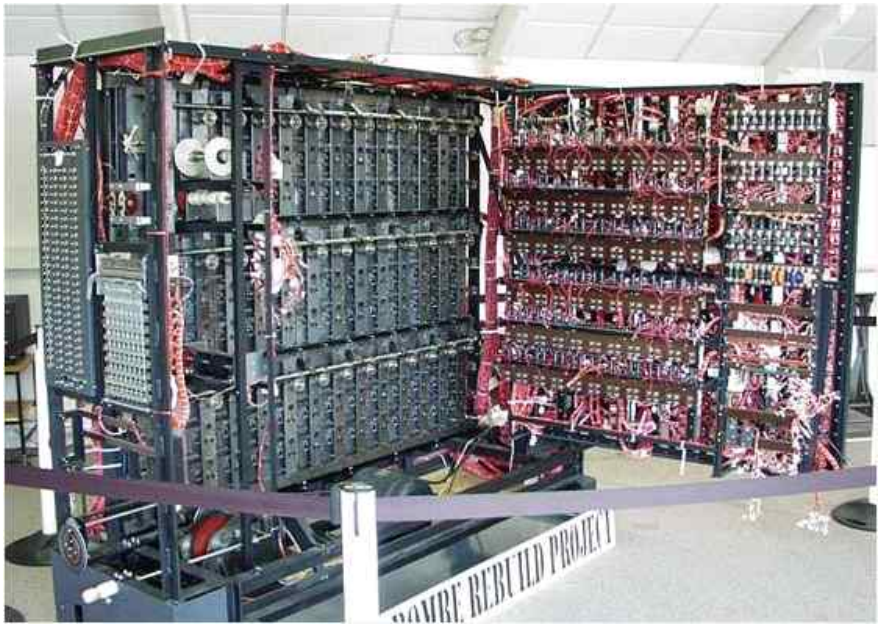
B) Accenture

C) Apple

D) BBC

Alan Turing committed suicide by eating a cyanide-laced apple in 1954. Rumor dispelled by Apple logo designer Rob Janoff.

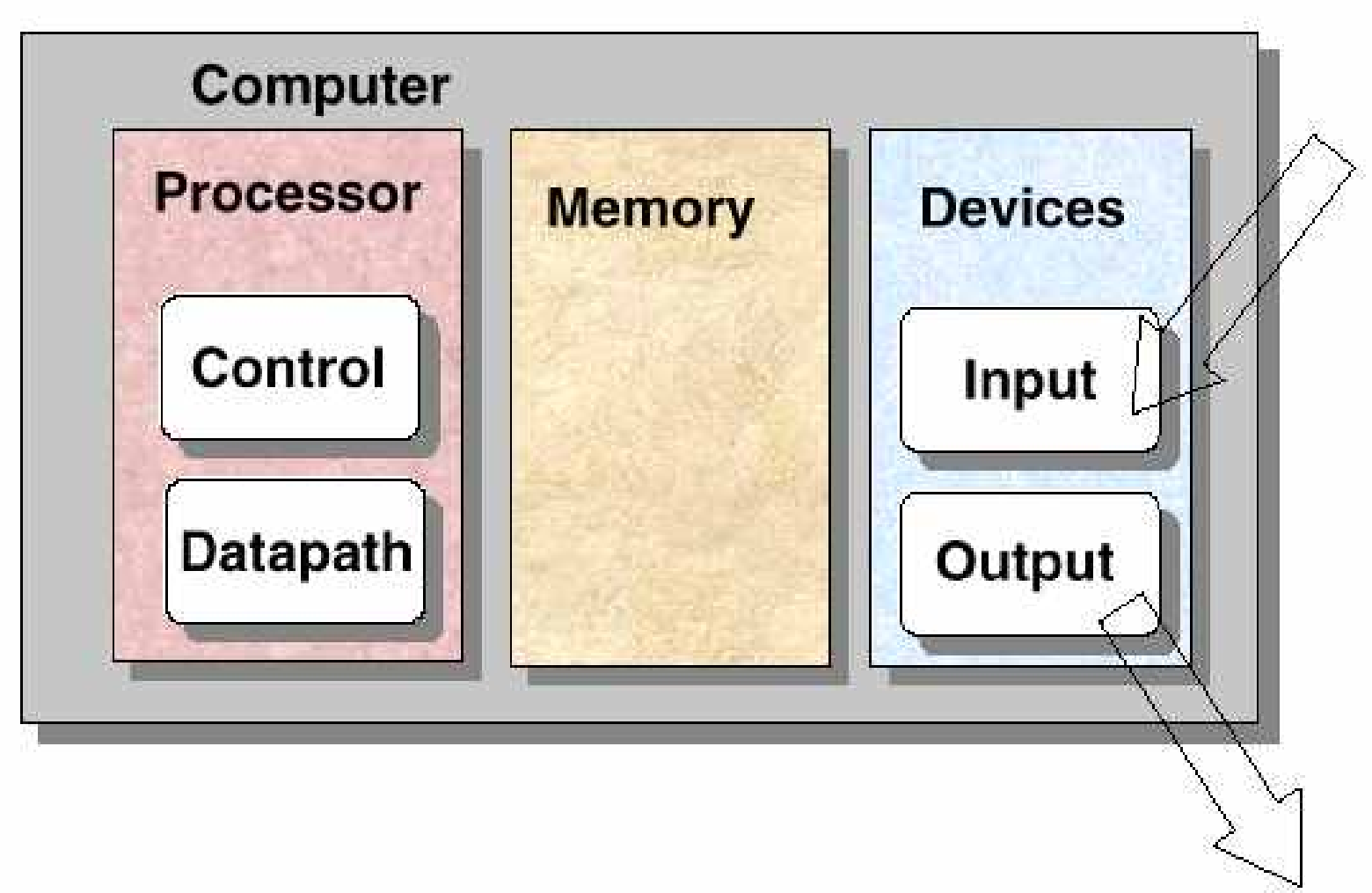
- Alan Turing (1912-1954)



A.I.
ARTIFICIAL INTELLIGENCE

AlphaGo

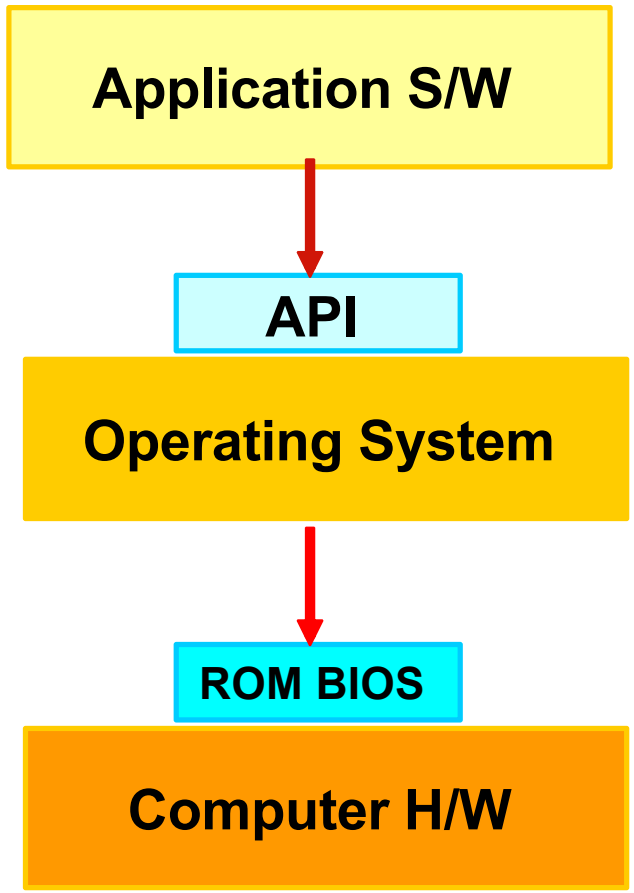
A Simple Picture



1-1 Digital Computers

- Digital – A limited number of discrete value
- Bit – A Binary Digit
- Program – A Sequence of instructions

- Computer = H/W + S/W
- Program(S/W)
  - A sequence of instruction
  - S/W = Program + Data
    - The data that are manipulated by the program constitute the data base
  - Application S/W
    - DB, word processor, Spread Sheet, game
  - System S/W
    - OS, Firmware, Compiler, Device Driver



- Computer Hardware
  - CPU
  - Memory
    - Read Only Memory (ROM)
    - Random Access Memory (RAM)
  - I/O Device
    - Input Device: Keyboard, Mouse, Scanner
    - Output Device: Printer, Plotter, Display
    - Storage Device(I/O): FDD, HDD, SSD

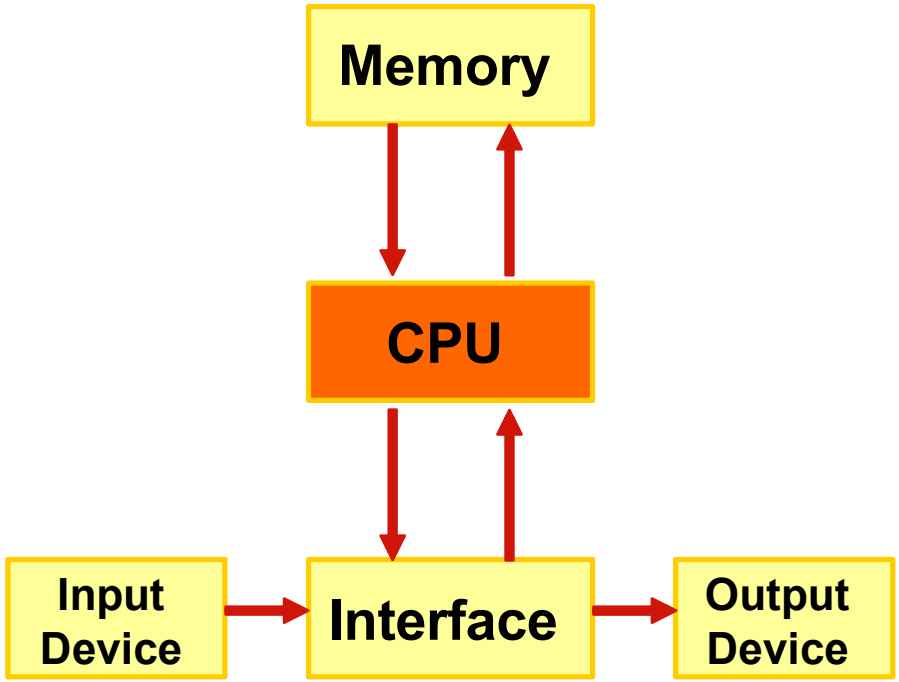


Figure 1-1 Block Diagram of a digital Computer

Example: Recipe

Program code
: Set of instructions

UI/UX

Data

Homemade Burgers

Ingredients

- 500g minced beef
- 3 spring onions
- 2 cloves of garlic
- 1 tsp paprika
- 1 tsp dried parsley
- black pepper

Method

Wash the spring onions. Chop them into small pieces with scissors. Throw away the roots.

Peel and crush the garlic.

Put all the ingredients into a bowl and mix them together with your hands.

Split the mixture into 4 equal pieces. Roll each piece into a ball and squash them to make burger shapes.

Grill the burgers for 15 minutes, turning once.

Eats Amazing
www.eatsamazing.co.uk

Ref) http://www.eatsamazing.co.uk/recipes-tutorials/cooking-with-small-child-homemade-burgers

1-1 Digital Computers

- What is “Computer Architecture”?

  - Hennessy and Patterson, *Computer Organization and Design(1990)*

  - Computer Architecture
    - Instruction Set Architecture (ISA)
    - Machine Organization
- “ISA”?
  - Instructions
  - Addressing modes
  - Instruction and data formats
  - Register
- “Machine Organization”?
  - CPU(Control, Data path), Memory, Input, Output

Class Overview

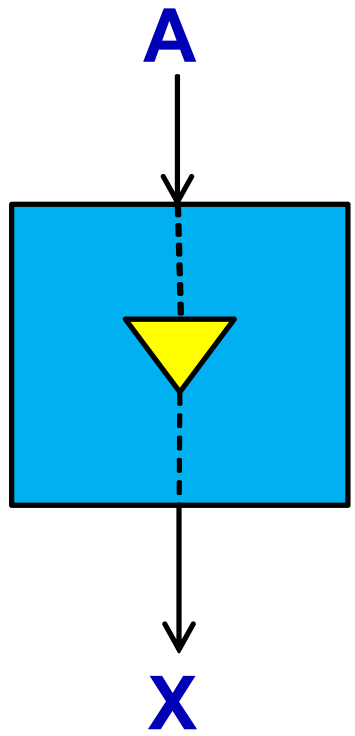
- Contents
  - Basic Computer Hardware Architecture
  - CPU, Memory, I/O System Design
  - Advanced Topics: Pipeline, Multiprocessor
- Related Subject
  - Digital Logic (논리설계), Discrete Mathematics (이산수학)
- Chapter Outline
  - Ch. 1 ~ Ch. 4: Digital Computer Hardware Architecture
  - Ch. 5 ~ Ch. 7: Basic Computer Design
  - Ch. 8 ~ Ch. 10: CPU Architecture and Pipeline Processing
  - Ch. 11 ~ Ch. 12: I/O and Memory Architecture
  - Ch. 13: Multiprocessor
- Homework
  - Solve the selected number of problems
  - Report of Survey for given theme
- Reference
  - Morris Mano, Computer Systems Architecture, Prentice-Hall, 3rd Ed.
  - John Hennessy, David Patterson, Computer Architecture: a Quantitative approach, Morgan Kaufmann. 3rd Ed.

1-2 Logic Gates

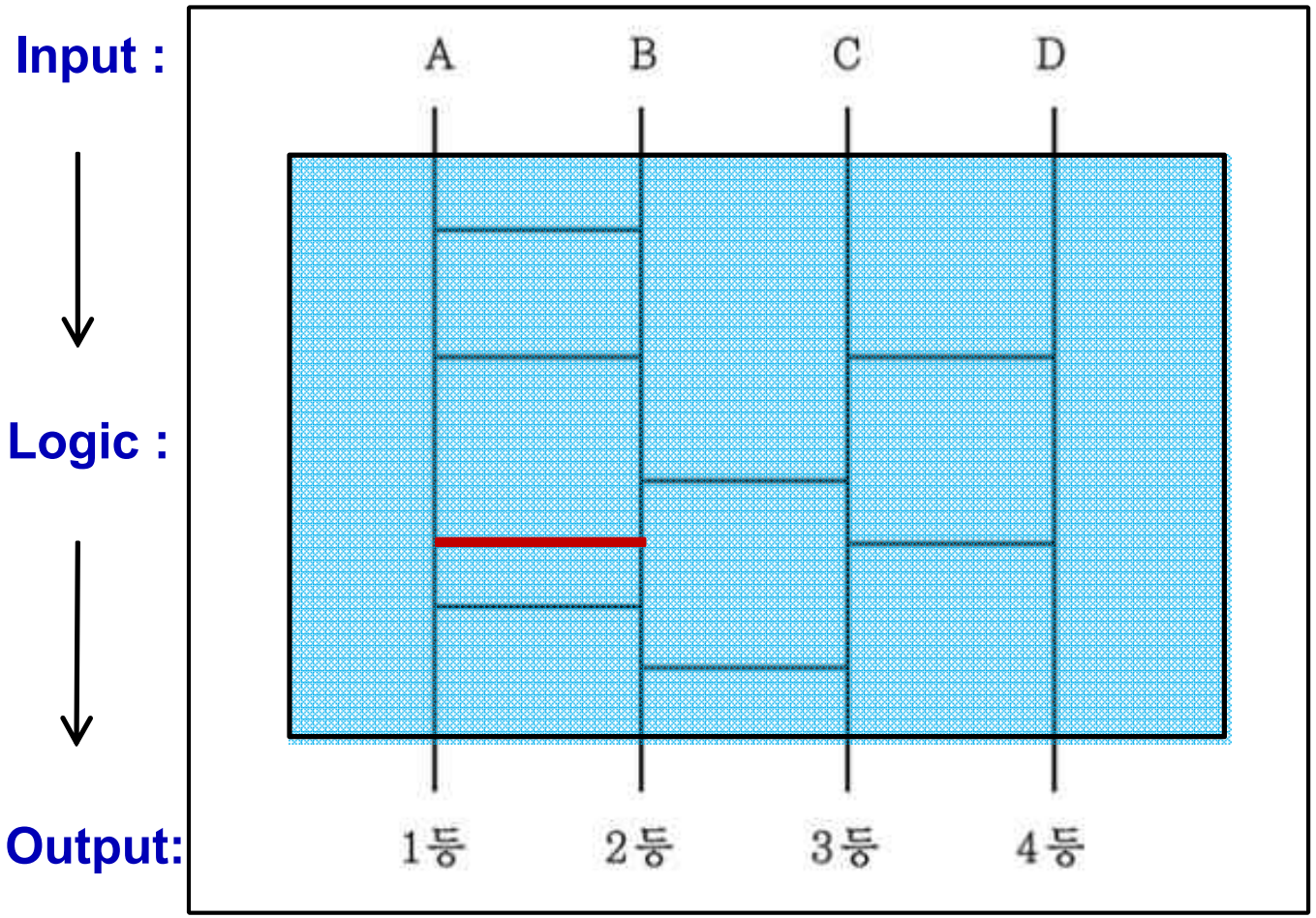
- ADC(Analog to Digital Conversion)
  - Signal → Physical Quantity → Binary Information
    V, A, F, 거리 → Discrete Value
    { 0 : 0.5 ↓ / 1 : 3 ↑
- Bit : 1/0
  - 2 bits : 00, 01, 10, 11 ; 4가지 = 2^2
  - 3 bits : 000, 001, 010, 011, 100, 101, 110, 111 ; 8가지 = 2^3
- Gate
  - The manipulation of binary information is done by logic circuit called "gate".
- *Fig. 1-2 Digital Logic Gates*
  - AND, OR, INVERTER, BUFFER, NAND, NOR, XOR, XNOR
- George Boole
  - 1815 ~ 1864

Boole Logic - Example

- What is it?
  - Input, Output 이 이진변수(binary variable)로 구성된 논리 회로
- 우주회 (雨酒會)
  - 비가 오면 → 술 먹는다
  - 비가 안 오면 → 술 안 먹는다
  - 비가 온다 : A = 1, 비가 안 온다 : A = 0
  - 술 먹는다 : X = 1, 술 안 먹는다 : X = 0
  - If (A == 1), Then X = 1; Else, X = 0;



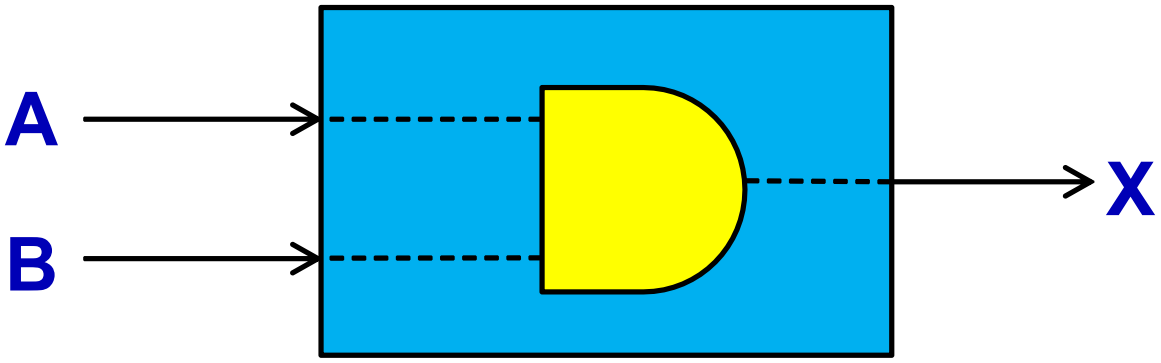
Boole Logic - Example



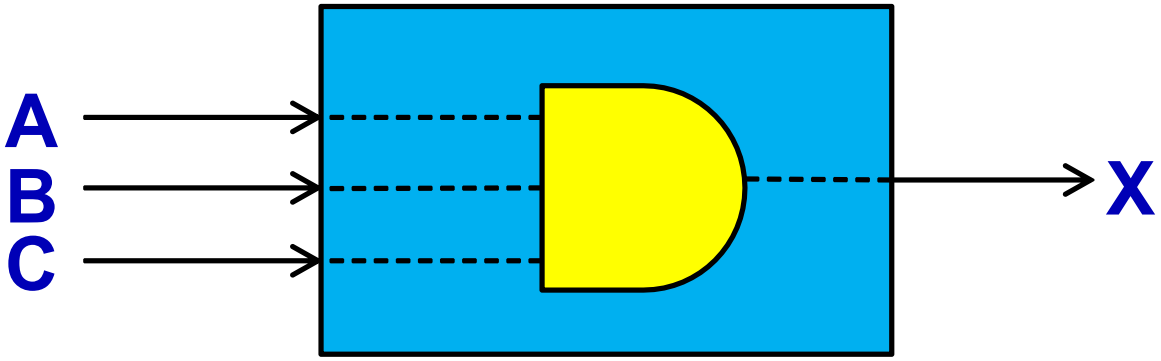
http://nogwon.blogspot.kr/2013/10/blog-post_2036.html

Boole Logic - Example

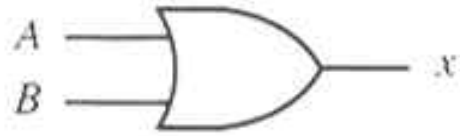
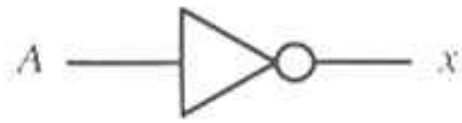
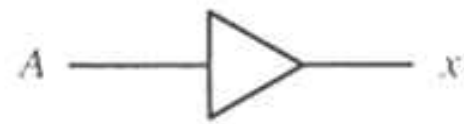
- 비(A)가 오고 친구(B)가 있으면 → 술(X) 먹는다
  - If (A==1)&&(B==1), Then X = 1; Else, X = 0;



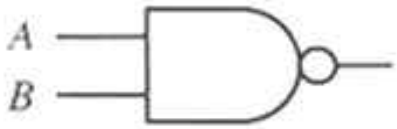
- 비(A)가 오고 친구(B)가 있고 돈(C)이 있으면 → 술(X) 먹는다
  - If (A==1)&&(B==1)&&(C==1), Then X = 1; Else, X = 0;



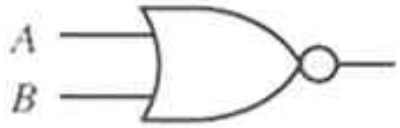
1-2 Logic Gates

Name	Graphic symbol	Algebraic function	Truth table
AND		$x = A \cdot B$ or $x = AB$	A B | x 0 0 | 0 0 1 | 0 1 0 | 0 1 1 | 1
OR		$x = A + B$	A B | x 0 0 | 0 0 1 | 1 1 0 | 1 1 1 | 1
Inverter		$x = A'$	A | x 0 | 1 1 | 0
Buffer		$x = A$	A | x 0 | 0 1 | 1

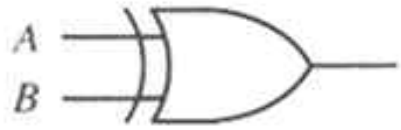
1-2 Logic Gates

NAND		$x = (AB)'$		

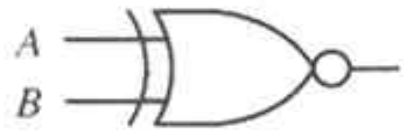
A	B	x
0	0	1
0	1	1
1	0	1
1	1	0

NOR		$x = (A + B)'$		

A	B	x
0	0	1
0	1	0
1	0	0
1	1	0

Exclusive-OR (XOR)		$x = A \oplus B$ or $x = A'B + AB'$		

A	B	x
0	0	0
0	1	1
1	0	1
1	1	0

Exclusive-NOR or equivalence		$x = (A \oplus B)'$ or $x = A'B' + AB$		

A	B	x
0	0	1
0	1	0
1	0	0
1	1	1

1-3 Boolean Algebra

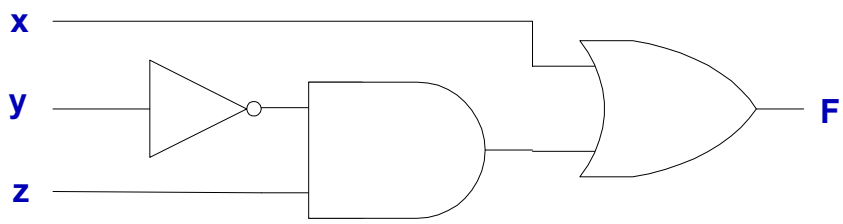
- Boolean Algebra
  - binary variable(A, B, x, y: T/F or 1/0) + logic operation(AND, OR, NOT…)
- Boolean Function: variable + operation
  - F(x, y, z) = x + y'z

- Truth Table: *Fig. 1-3(a)*
  Relationship between a function and variable

- Logic Diagram: *Fig. 1-3(b)*
  Algebraic Expression ⇒
  Logic Diagram(gates로 표현)

2^n Combination
Variable n = 3

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1



1-3 Boolean Algebra

- Purpose of Boolean Algebra
  - To facilitate the analysis and design of digital circuit
- Convenient Tools
  - Truth table : relationship between binary variables
  - Logic diagram : input-output relationship
  - Find simpler circuits for the same function
- Boolean Algebra Rule : Tab. 1-1

- Operation with 0 and 1: **x + 0 = x , x + 1 = 1 , x • 1 = x , x • 0 = 0**

- Idempotent Law: **x + x =x , x • x = x**

- Complementary Law: **x + x' = 1 , x • x' = 0**

- Commutative Law: **x + y = y + x , x • y = y • x**

- Associative Law: **x + (y + z) = (x + y) + z , x • (y • z) = (x • y) • z**

- Distributive Law: **x • (y+ z) = (x • y) + (x • z) , x + (y • z) = (x + y) • (x + z)**

- DeMorgan's Law: **(x + y)' = x' • y' , (x • y)' = x' + y'**

General Form: $(x_1 + x_2 + x_3 + \dots x_n)' = x_1' \cdot x_2' \cdot x_3' \cdot \dots x_n'$

$(x_1 \cdot x_2 \cdot x_3 \cdot \dots x_n)' = x_1' + x_2' + x_3' + \dots x_n'$

$(A \cup B)^c = A^c \cap B^c$

1-3 Boolean Algebra

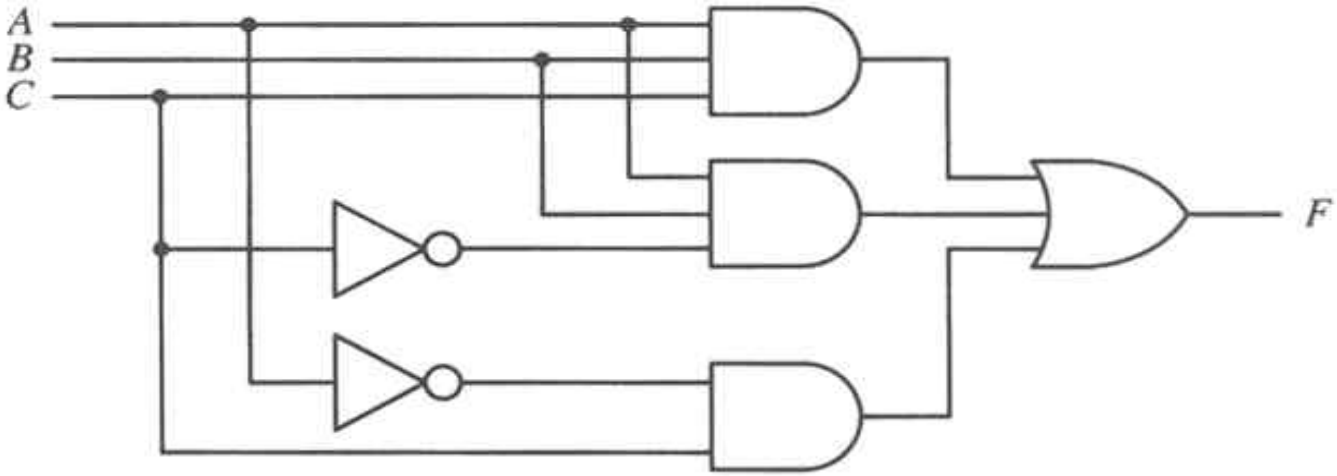
- [예 제]
  - F= AB’ + C’D + AB’ + C’D
    = x + x (let x= AB’ + C’D)
    = x
    = AB’ + C’D

- [예 제]
  - F= ABC + ABC’ + A’C
    = AB(C + C’) + A’C
    = AB + A’C

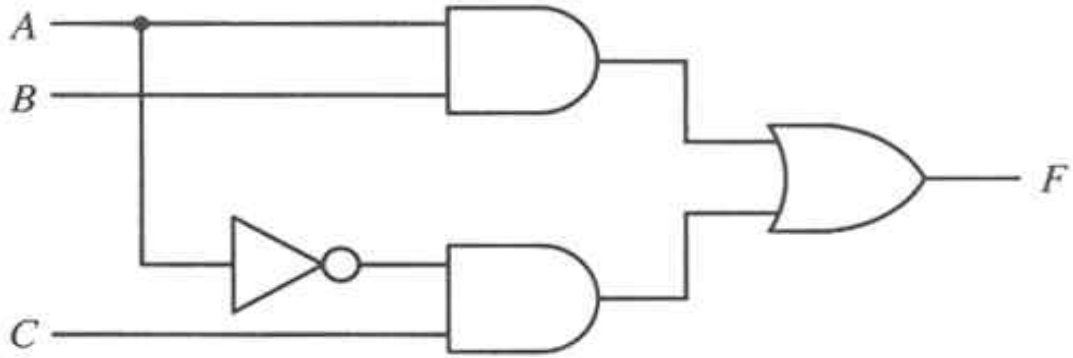
Fig. 1-6(a)

Fig. 1-6(b)

1 inverter, 1 AND gate 감소



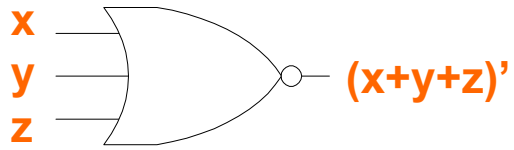
(a) $F = ABC + ABC' + A'C$



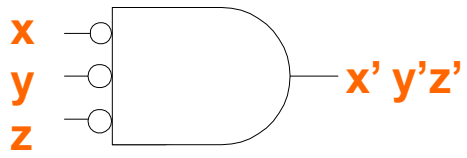
(B) $F = AB + A'C$

Figure 1-6 Two logic diagrams for the same Boolean function.

- *Fig. 1-4 2 graphic symbols for NOR gate*

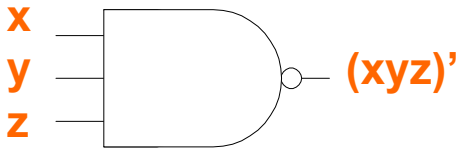


(a) OR-invert

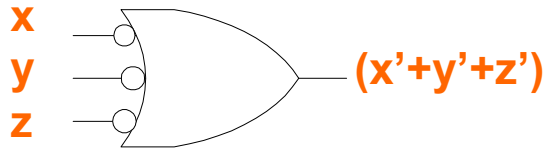


(b) invert-OR

- *Fig. 1-5 2 graphic symbols for NAND gate*



(a) NAND-invert



(b) invert-NAND

1-4 Map Simplification

- Karnaugh Map(K-Map)
  - Map method for simplifying Boolean expressions
- Minterm / Maxterm
  - Minterm : n variables ***product*** (x=1, x'=0)
  - Maxterm : n variables ***sum*** (x=0, x'=1)
- 2 variables example

x	y	Minterm		Maxterm	
0	0	x'y'	m_0	x + y	M_0
0	1	x'y	m_1	x + y'	M_1
1	0	x y'	m_2	x'+ y	M_2
1	1	x y	m_3	x'+ y'	M_3

$m_0 + m_1 + m_2 + m_3$

$M_0 \bullet M_1 \bullet M_2 \bullet M_3$

- F = x'y + xy

  m_1 m_3

  $= \Sigma(1,3)$ ($m_1 + m_3$)

  $= \Pi(0,2)$ (Complement = $M_0 \bullet M_2$)

1-4 Map Simplification

- Map
  - 2 variables

A \ B	0	1
0	0	1
1	2	3

  - 3 variables

A \ BC	00	01	11	10
0	0	1	3	2
1	4	5	7	6

  - 4 variables

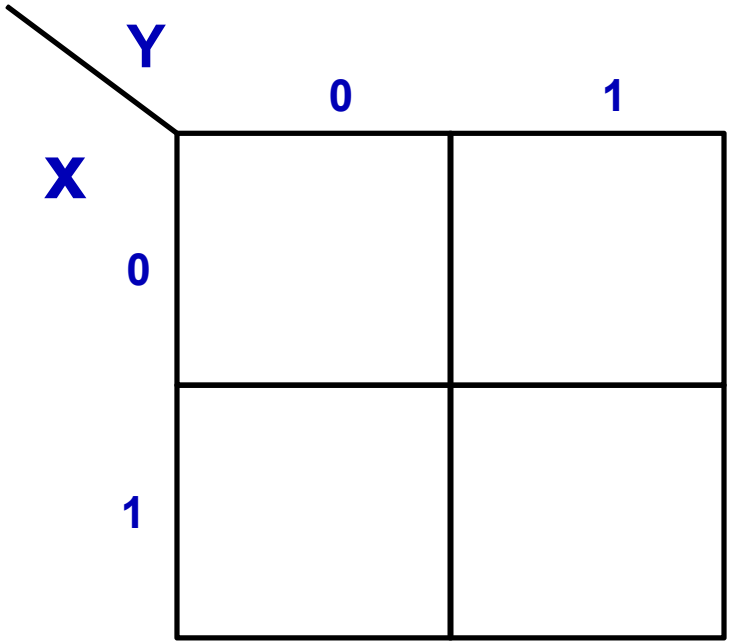
AB \ CD	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

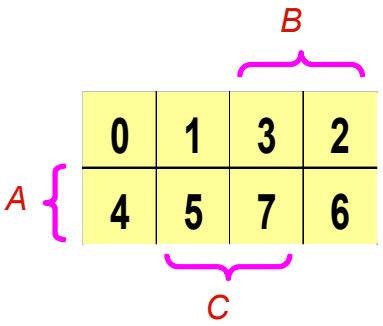
  - 5 variables

A \ BCDE	000	001	011	010	110	111	101	100
00	0	1	3	2	6	7	5	4
01	8	9	11	10	14	15	13	12
11	24	25	27	26	30	31	29	28
10	16	17	19	18	22	23	21	20

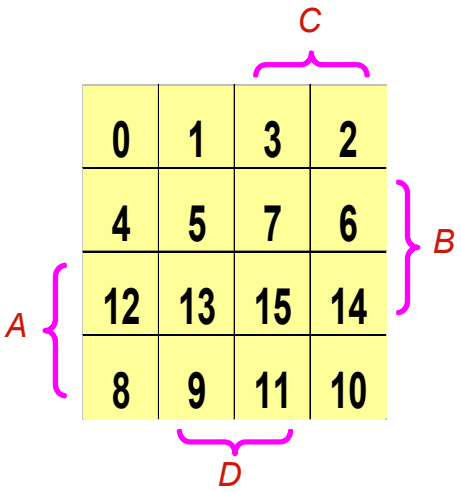
Fig 1-7 Maps for two-, three-, four-, and five-variable functions

0	1
2	3





YZ / X	00	01	10 (11)	11 (10)
0	000	001	011	010
1	100	101	111	110



AB \ CD	00	01	11	10
00				
01				
11				
10				

1-4 Map Simplification

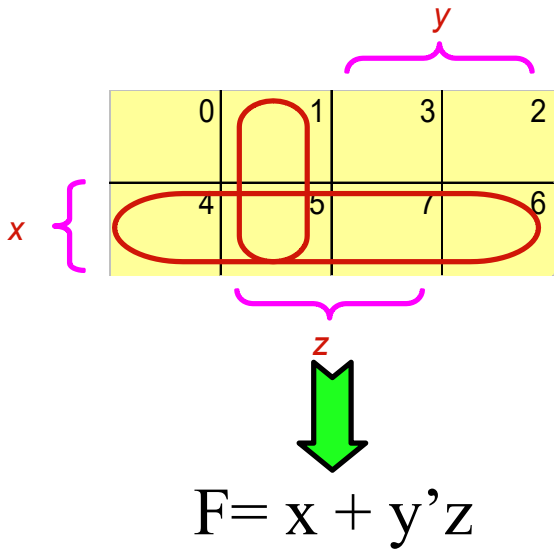
- [예제] F= x + y'z

(1) Truth Table

x	y	z	F	Minterm
0	0	0	0	m_0
0	0	1	1	m_1
0	1	0	0	m_2
0	1	1	0	m_3
1	0	0	1	m_4
1	0	1	1	m_5
1	1	0	1	m_6
1	1	1	1	m_7

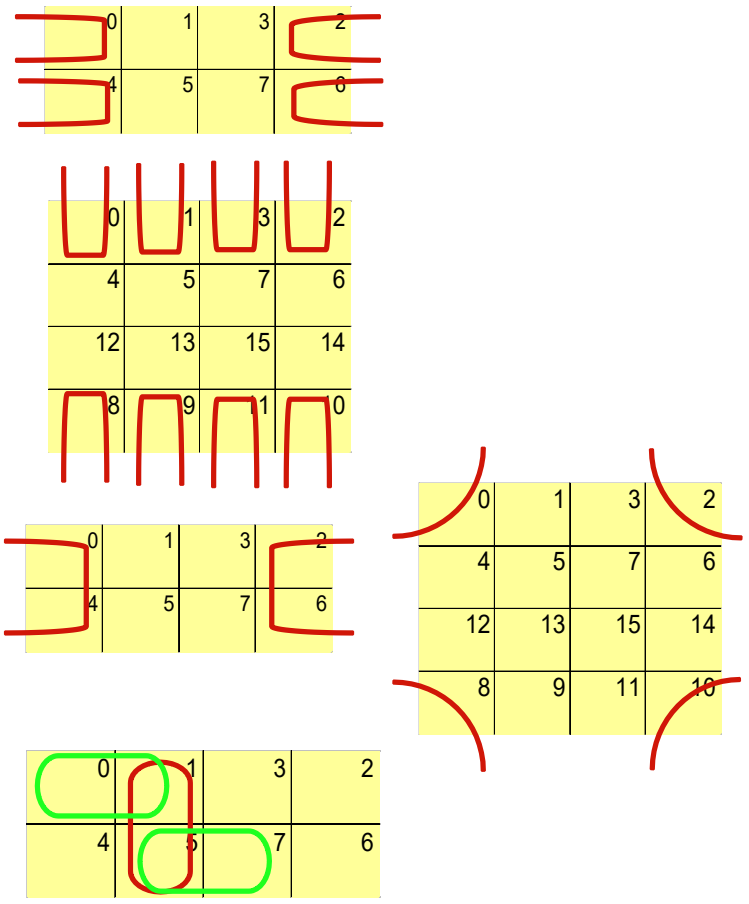
(2) $F(x, y, z) = \Sigma(1,4,5,6,7)$

(3) Karnaugh Map



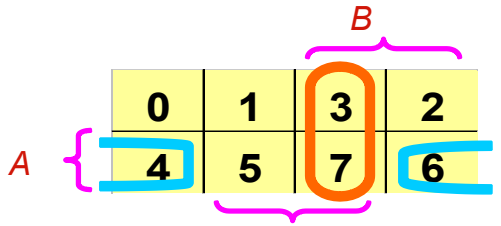
1-4 Map Simplification

- Adjacent Square
  - Number of square = 2^n (2, 4, 8, ….)
  - The squares at the extreme ends of the same horizontal row are to be considered adjacent
  - The same applies to the top and bottom squares of a column
  - The four corner squares of a map must be considered to be adjacent
  - Groups of combined adjacent squares may share one or more squares with one or more group

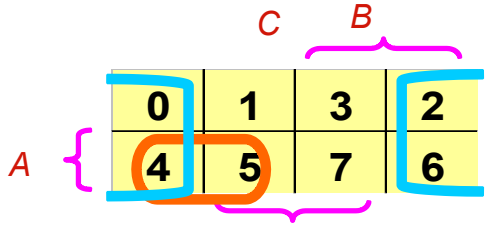


1-4 Map Simplification

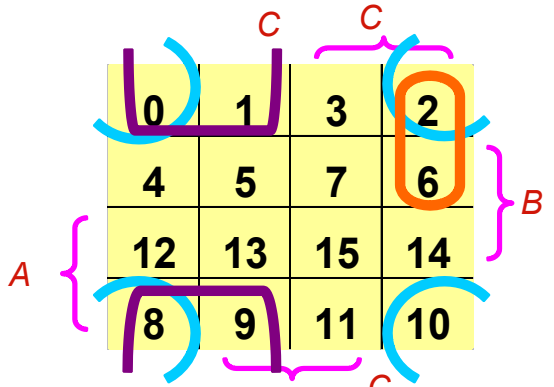
- [예제] $F(A,B,C)=\Sigma(3,4,6,7)$
  - F=AC' + BC



- [예제] $F(A,B,C)=\Sigma(0,2,4,5,6)$
  - F=C' + AB'



- [예제] $F(A,B,C,D)=\Sigma(0,1,2,6,8,9,10)$
  - F=B'C' + B'D' + A'CD'



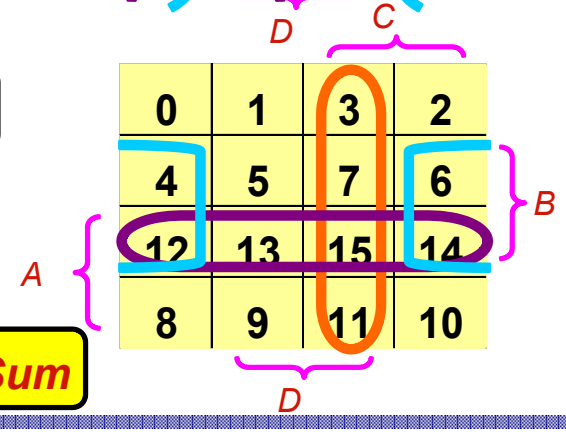
- Product-of-Sums Simplification

$F(A,B,C,D)=\Sigma(0,1,2,5,8,9,10)$

F=B'D' + B'C' + A'C'D — **Sum of product**

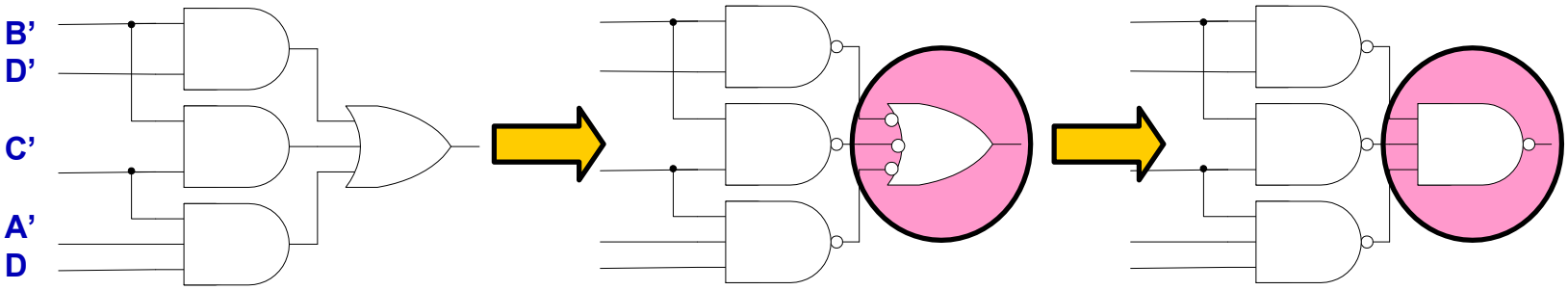
F'=AB + CD + BD'(square marked 0's)

F''(F)=(A' + B')(C' + D')(B' + D) — **Product of Sum**

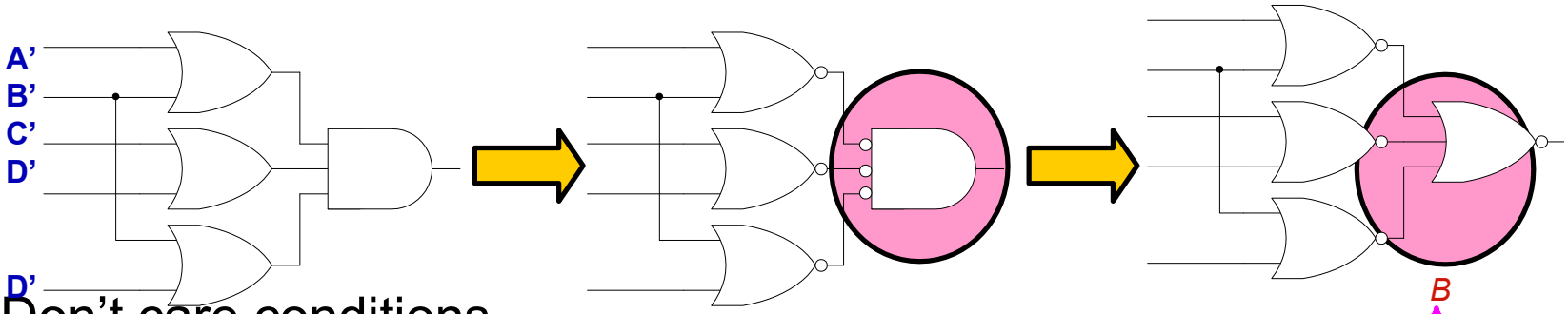


1-4 Map Simplification

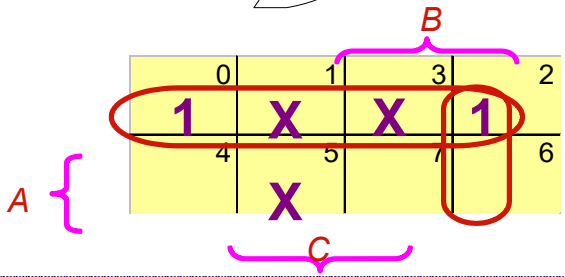
- NAND Implementation
  - Sum of Product : F=B'D' + B'C' + A'C'D



- NOR Implementation
  - Product of Sum : F=(A' + B')(C' + D')(B' + D)



- Don't care conditions
  - F(A,B,C)=Σ(0, 2, 6), d(A,B,C)= Σ(1, 3, 5)
  - F=A' + BC'= Σ(0, 1, 2, 3, 6)



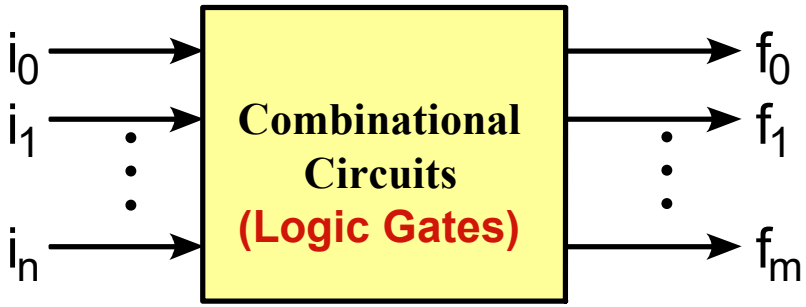
조합회로 vs. 순차회로

- Combinational circuit (조합회로)
  - Input 값에 의해서만 Output 결정
  - ex) 덧셈기(adder), 우주회(雨酒會)
  - Logic gate로 회로 구성
- Sequential circuit (순차회로)
  - Inout 값과 상태(state)에 의해 Output 결정
  - Counter
  - Logic Gate + FlipFlop으로 회로 구성

- 1-5: 조합회로
- 1-6: 플립플롭
- 1-7: 순차회로

1-5 Combinational Circuits

- Combinational Circuits
  - A connected arrangement of *logic gates* with a set of inputs and outputs
  - *Fig. 1-15 Block diagram of a combinational circuit*



- Analysis
  - Logic circuits diagram → Boolean function or Truth table
- Design(Analysis의 반대)
  - 1. The Problem is stated
  - 2. I/O variables are assigned
  - 3. Truth table(I/O relation)
  - 4. Simplified Boolean Function(Map 과 Boolean 대수 이용)
  - 5. Logic circuit diagram

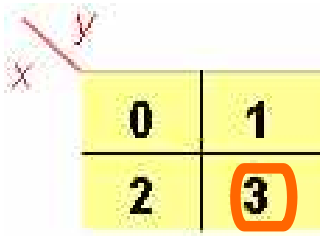
Experience

1-5 Combinational Circuits

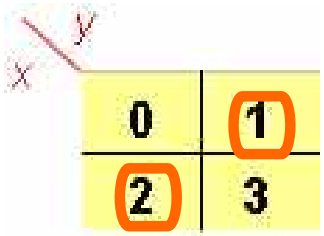
- Design Example : Half Adder
  - 1. Half adder is a combinational circuits that forms the arithmetic sume of two input bit
  - 2. 2 Input(x, y), 2 Output(S: sum, C: carry)
  - 3. Truth Table

Input		Output	
x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

  - 4. Simplification



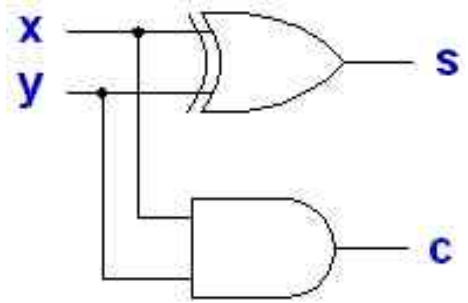
C= xy



S= x'y + xy'

=x ⊕ y

  - 5. Logic circuit diagram



  - 6. Block diagram

Half Adder (반가산기) 문제점


```
    11          3
+   01      +   1
------      -----
    10          2
```

→ **Wrong**

```
     1
    11          3
+   01      +   1
------      -----
   100          4
```

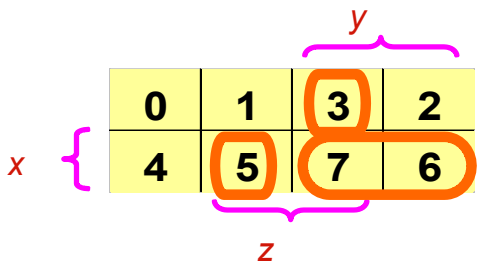
→ **Correct**

1-5 Combinational Circuits

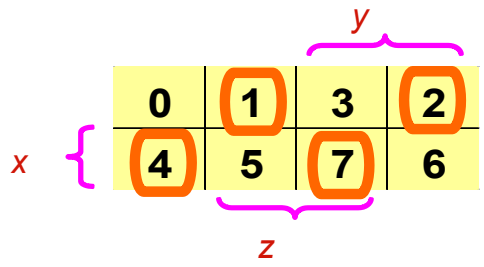
- Design Example : Full Adder
  - 1. Full adder is a combinational circuits that forms the arithmetic sume of three input bit(Carry considered)
  - 2. 3 Input(x, y, z), 2 Output(S: sum, C: carry)
  - 3. Truth Table

Input			Output	
x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

  - 4. Simplification



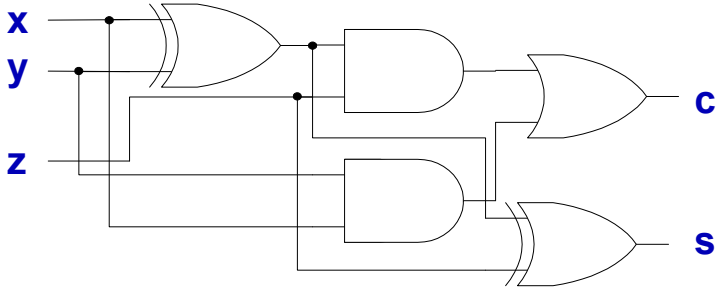
C= xy'z + x'yz + xy
=z(xy' + x'y) + xy
=z(x ⊕ y) + xy



S=xy'z' + x'y'z + xyz + x'yz'
= z'(xy' + x'y) + z(x'y' + xy)
= z'(x ⊕ y) + z(x ⊕ y)'
=a'b + ab' (let a=z, b=x ⊕ y)
=x ⊕ y ⊕ z

(x ⊕y)'=(xy'+x'y)'
=(x'+y)(x+y')
=xx'+x'y'+xy+yy'
=x'y'+xy

  - 5. Logic circuit diagram



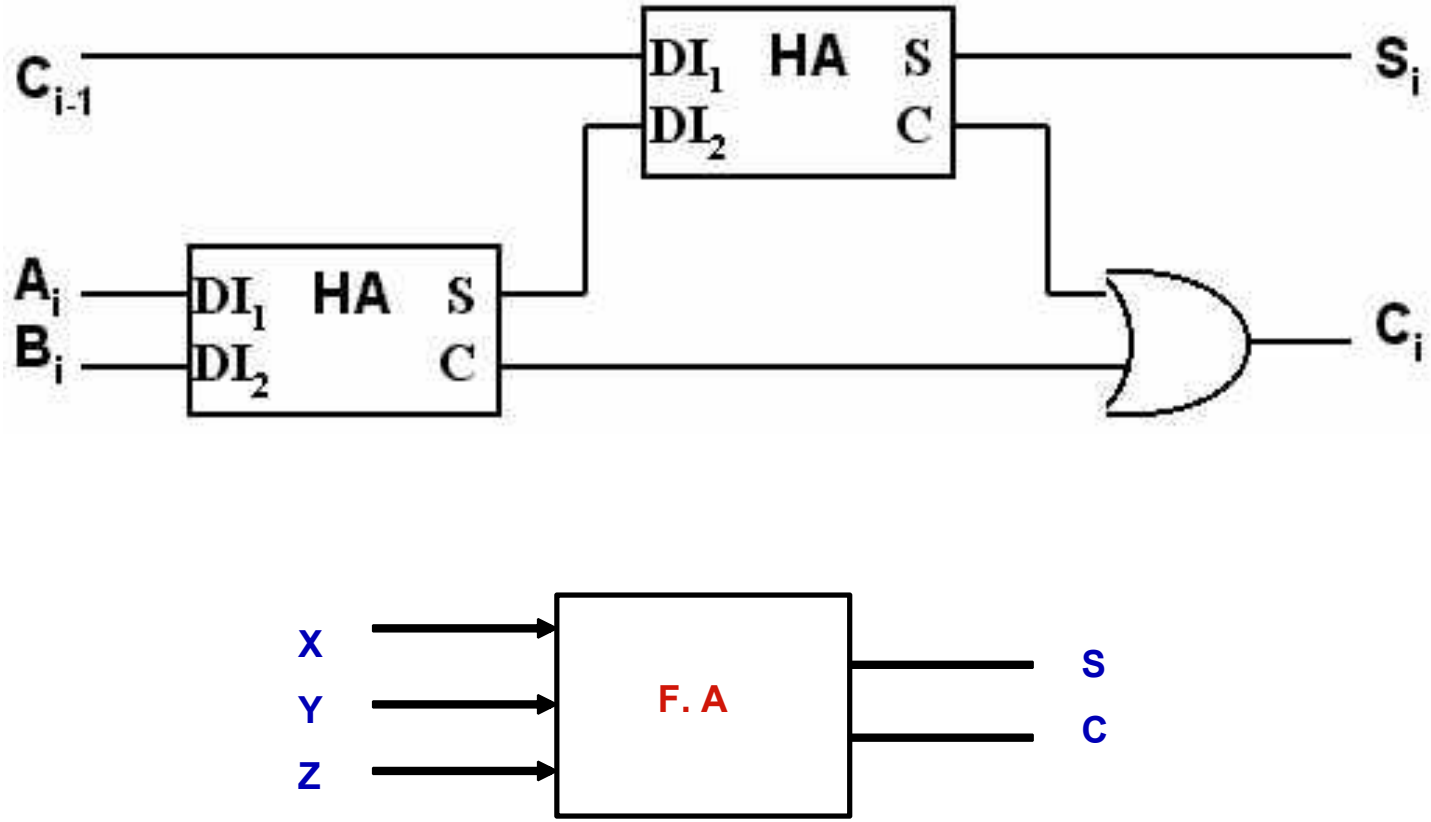


Fig 1-18. Block diagram

조합회로 vs. 순차회로

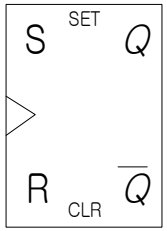
- Combinational circuit (조합회로)
  - Input 값에 의해서만 Output 결정
  - ex) 덧셈기(adder), 우주회(雨酒會)
  - Logic gate로 회로 구성
- Sequential circuit (순차회로)
  - Input 값과 상태(state)에 의해 Output 결정
  - Counter
  - Logic Gate + FlipFlop으로 회로 구성

- 1-5: 조합회로
- 1-6: 플립플롭
- 1-7: 순차회로

1-6 Flip-Flops

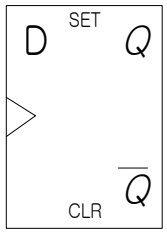
Combinational Circuit = Gate
Sequential Circuit = Gate + F/F

- Flip-Flop
  - The *storage elements* employed in clocked *sequential circuit*
  - A binary cell capable of storing one bit of information
- SR*(Set/Reset)* F/F



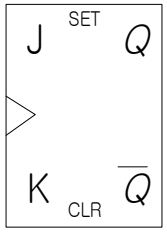
S	R	Q(t+1)	
0	0	Q(t)	no change
0	1	0	clear to 0
1	0	1	set to 1
1	1	?	Indeterminate

- D*(Data)* F/F



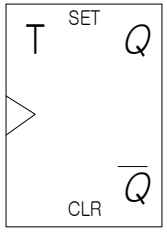
D	Q(t+1)	
0	0	clear to 0
1	1	set to 1

  - "no change" condition이 없다 : Q(t+1)=D
    - 해결방법 : 1) Disable Clock
      2) Feedback output into input
- JK*(Jack/King)* F/F



J	K	Q(t+1)	
0	0	Q(t)	no change
0	1	0	clear to 0
1	0	1	set to 1
1	1	Q(t)'	Complement

  - JK F/F is a refinement of the SR F/F
  - The indeterminate condition of the SR type is defined in complement
- T*(Toggle)* F/F



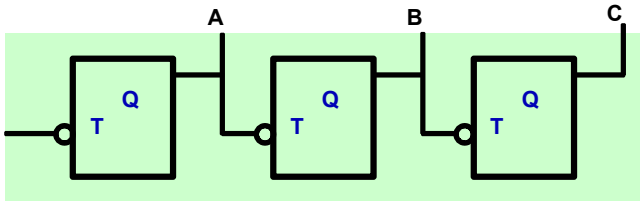
T	Q(t+1)	
0	Q(t)	no change
1	Q'(t)	Complement

  - T=1(J=K=1), T=0(J=K=0) 이면 JK F/F
  - 수식 표현 : Q(t+1)= Q(t) ⊕ T

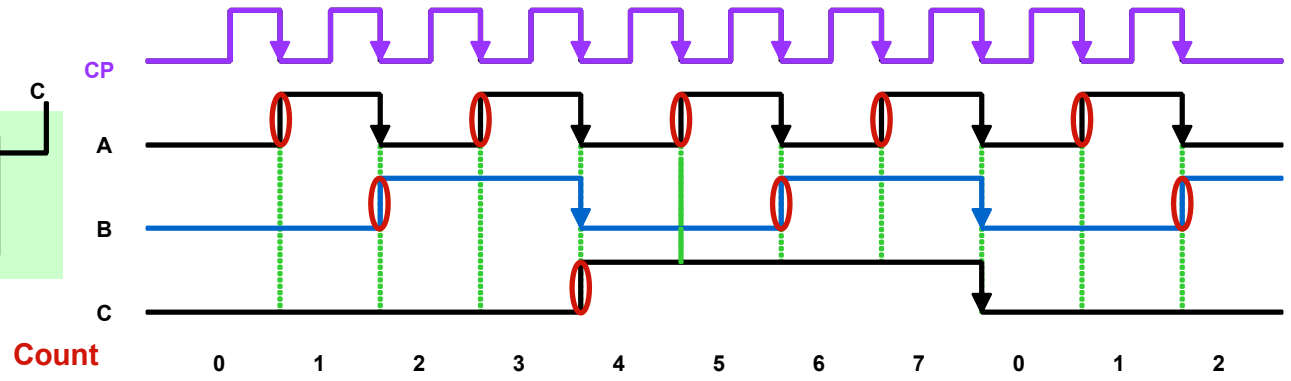
Clock Speed

- 1 Hz – 1/sec
- 1 KHz – 1,000 (천)/sec
- 1 MHz – 1,000,000 (백만)/sec
- 1 GHz – 1,000,000,000 (십억)/sec
- 1 THz – 1,000,000,000,000 (조)/sec
- 1 PHz – 1,000,000,000,000,000 (천조)/sec

- 2GHz clock speed: 초당 20억번 clock 생성 -> 1 clock 당 0.5 ns



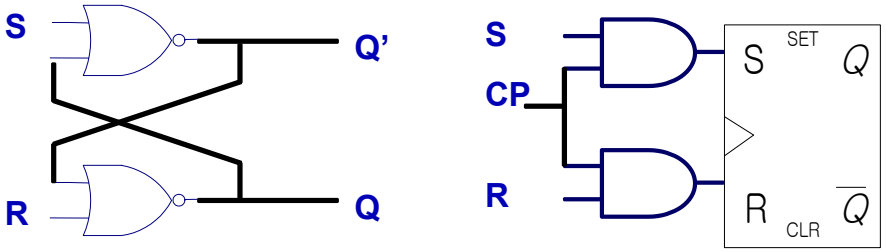
(a) Consists of T Flip-Flop



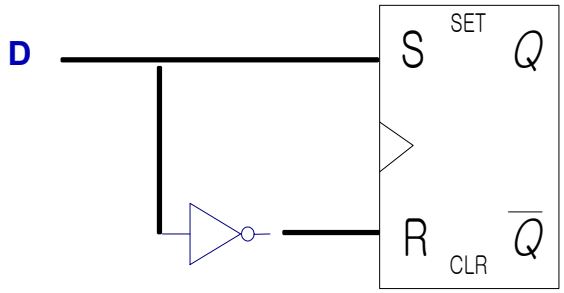
(b) Timing chart

1-6 Flip-Flops

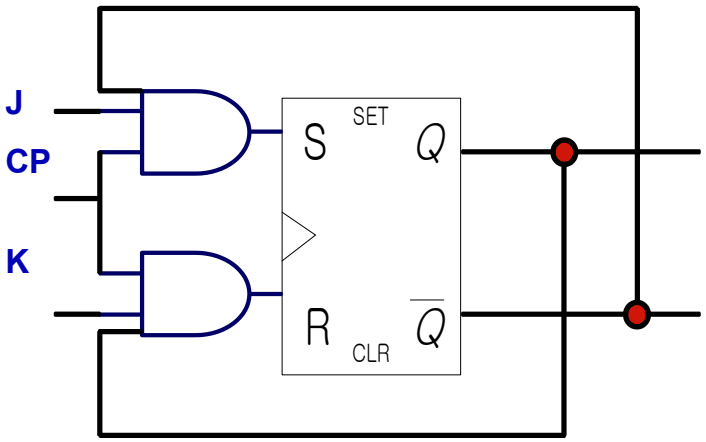
- SR*(Set/Reset)* F/F



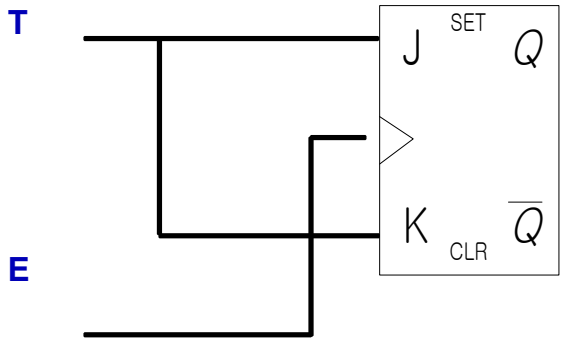
- D*(Data)* F/F



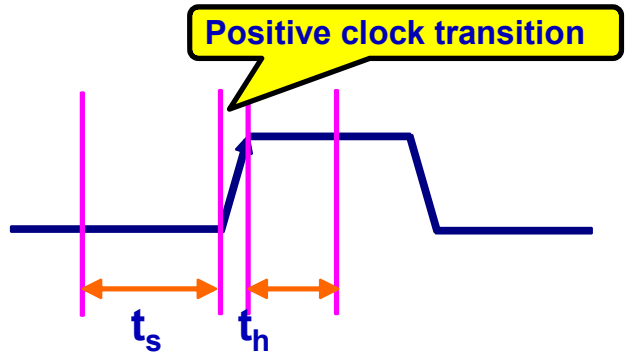
- JK*(Jack/King)* F/F



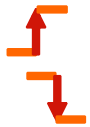
- T*(Toggle)* F/F



1-6 Flip-Flops



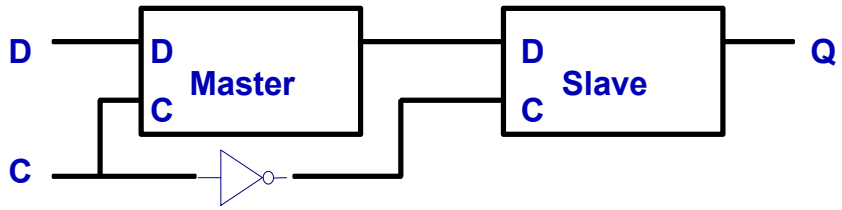
- Edge-Triggered F/F
  - State Change : *Clock Pulse*
    - Rising Edge(positive-edge transition)
    - Falling Edge(negative-edge transition)



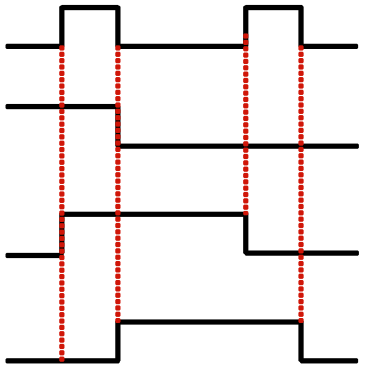
  - Setup time(20ns)
    - minimum time that D input must remain at constant value before the transition.
  - Hold time(5ns)
    - minimum time that D input must not change after the positive transition.
  - Propagation delay(max 50ns)
    - time between the clock input and the response in Q
    - 일반 논리 gate에서는 2-20 ns이며 setup 및 hold time은 F/F에서만 정의되며 일반 논리 gate에서는 정의되지 않음.
  - Master-Slave F/F
    - 2개의 F/F을 사용(Slave 와 Master F/F)하며 negative-edge transition 사용
    - 위와 같이 사용하는 이유: **Race 현상**을 방지

1-6 Flip-Flops

- Master – Slave D*(Data)* F/F



a. 논리도



b. 파형도

1-6 Flip-Flops

- Race 현상
  - 조건 - Setup time > Propagation delay
  - 증상 - 0 과 1을 반복하다가 Unstable한 상태가 된다
  - 해결책 - Edge triggered F/F 또는 Master/Slave F/F 사용
  - 예제
    - 7470 : J-K Edge triggered F/F
    - 7471 : J-K Master/Slave F/F
- Excitation Table
  - Required input combinations for a given change of state
  - Present State 와 Next State로 표현

SR F/F

Q(t)	Q(t+1)	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

JK F/F

Q(t)	Q(t+1)	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

D F/F

Q(t)	Q(t+1)	D
0	0	0
0	1	1
1	0	0
1	1	1

T F/F

Q(t)	Q(t+1)	T
0	0	0
0	1	1
1	0	1
1	1	0

Don't Care

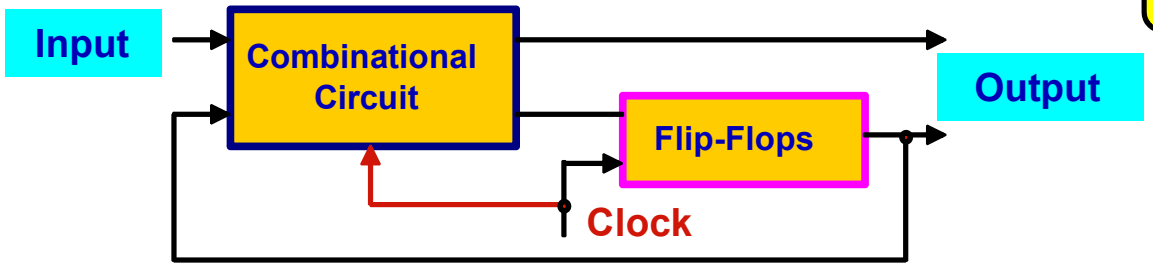
1 : Set to 1
0 : Complement

1 : Clear to 0
0 : No change

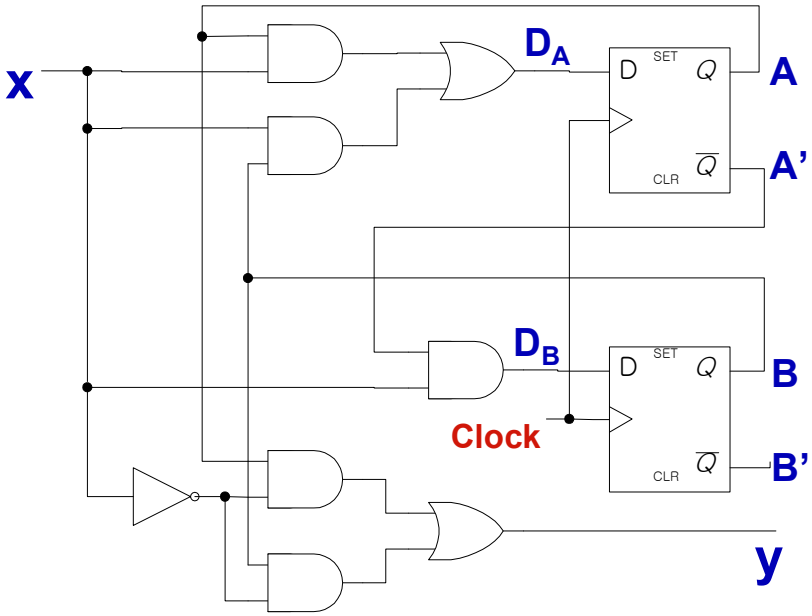
1-7 Sequential Circuits

- A sequential circuit is an interconnection of F/F and Gate
- Clocked synchronous sequential circuit

Combinational Circuit = Gate
Sequential Circuit = Gate + F/F



- Flip-Flop Input Equation
  - Boolean expression for F/F input
  - Input Equation 예제
    - $D_A = Ax + Bx$, $D_B = A'x$
  - Output Equation
    - $y = Ax' + Bx'$
  - *Fig. 1-25 Example of a sequential circuit*



1-7 Sequential Circuits

State Table

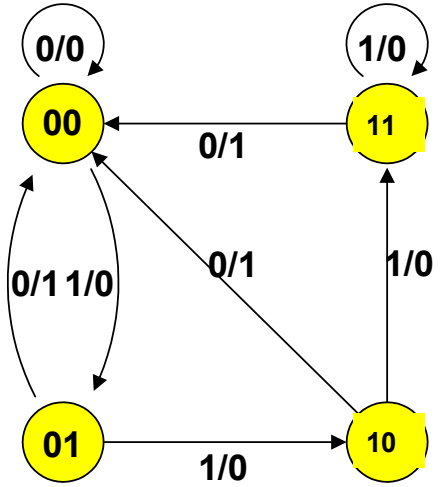
- Present state, input, next state, output 표현

Input Equ. = Next State

Present State		Input			Input Equ.		Next State		Output
A	B	x	Ax	Bx	D_A	D_B	A	B	y
0	0	0	0	0	0	0	0	0	0
0	0	1	0	0	0	1	0	1	0
0	1	0	0	0	0	0	0	0	1
0	1	1	0	1	1	1	1	1	0
1	0	0	0	0	0	0	0	0	1
1	0	1	1	0	1	0	1	0	0
1	1	0	0	0	0	0	0	0	1
1	1	1	1	1	1	0	1	0	0

State Diagram

- Graphical representation of state table
  - Circle(state), Line(transition), I/O(input/output)



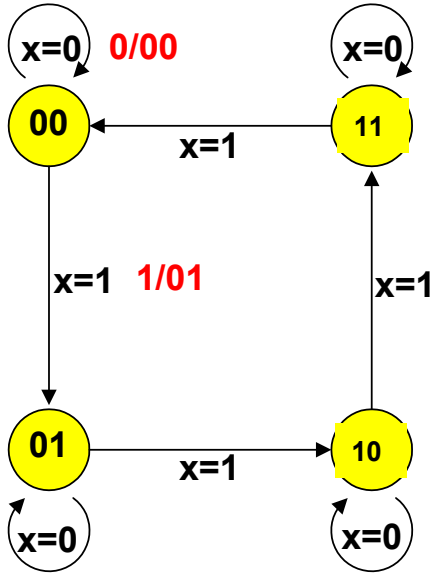
JK F/F			
Q(t)	Q(t+1)	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

Next State = Output

Design Example: Binary Counter

- x=1: 00, 01, 10, 11, 00, 01, …..
  x=0: no change
- State Diagram:

4 state(00, 01, 10, 11)

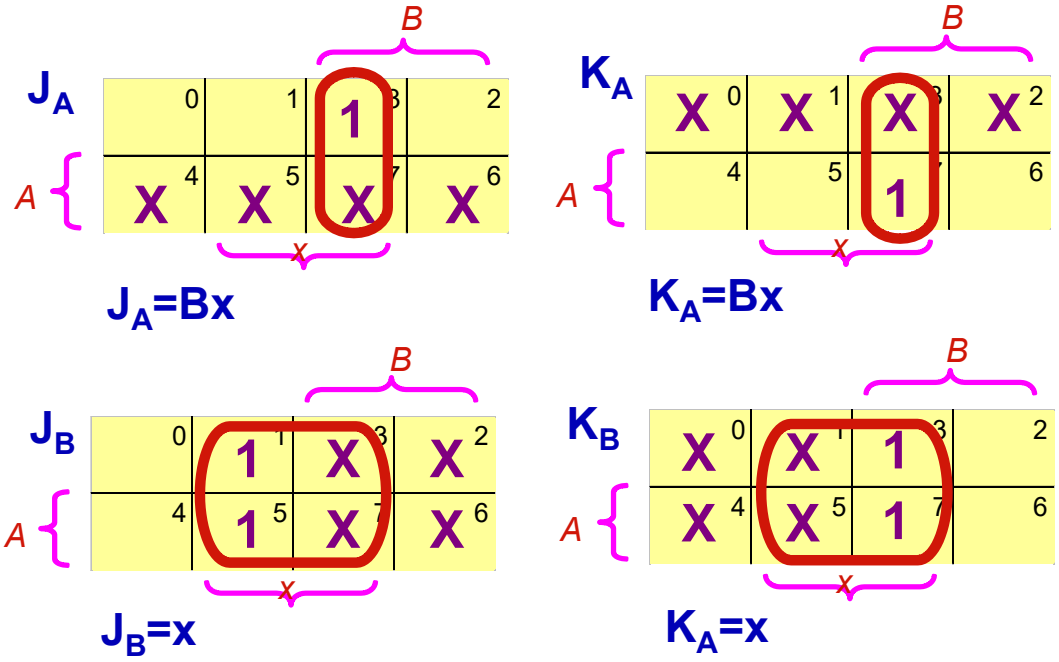


- Excitation Table(2 bit counter = 2 F/F)

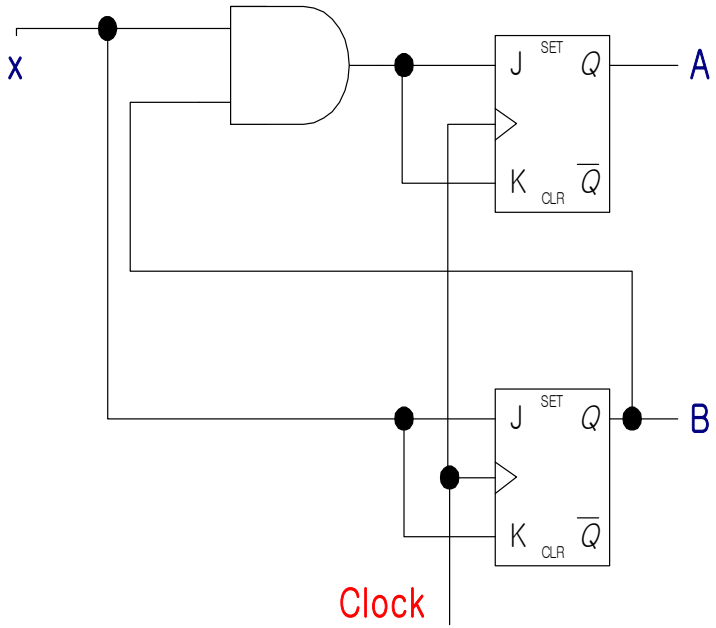
Present State		Input	Next State		F/F Input			
A	B	x	A	B	J_A	K_A	J_B	K_B
0	0	0	0	0	0	x	0	x
0	0	1	0	1	0	x	1	x
0	1	0	0	1	0	x	x	0
0	1	1	1	0	1	x	x	1
1	0	0	1	0	x	0	0	x
1	0	1	1	1	x	0	1	x
1	1	0	1	1	x	0	x	0
1	1	1	0	0	x	1	x	1

1-7 Sequential Circuits

- Map for simplification
  - Input variable: A, B, x



- Logic Diagram



- Sequential Circuit Design Procedure
  - 1-5 절 참고(Combinational Circuit Design)
  - Sequential Circuit은 절차 3에서 State diagram및 State table 이용
  - F/F 수: 2^{m+n} (m - State 수, n - Input 수)

1. The Problem is stated
2. I/O variables are assigned
3. Truth table(I/O relation)
4. Simplified Boolean Function
5. Logic circuit diagram